



Name : Yeamin Rahman Mahid
ID: 2023-3-60-375

Submitted to:
Md. Adnan Morshed
Lecturer,
Department of Computer Science
And Engineering

1. Generate a binary mask of the tumor from Figure 1 using any segmentation method of your choice, then apply:

i. Morphological Dilation

ii. Morphological Erosion

By using appropriate structuring elements on the mask.

```
# =====  
# PROBLEM 1 - Tumor mask + dilation + erosion  
# =====  
def make_tumor_mask(image):  
    # Contrast normalization  
    lo, hi = np.percentile(image, (2, 98))  
    enhanced = exposure.rescale_intensity(image, in_range=(lo, hi))  
  
    # Otsu threshold  
    threshold = filters.threshold_otsu(enhanced)  
    binary = enhanced > threshold  
  
    # Basic cleanup  
    binary = morphology.remove_small_objects(  
        binary, min_size=max(30, int(image.size * 0.0002))  
    )  
    binary = morphology.binary_opening(binary, disk(2))  
    binary = morphology.binary_closing(binary, disk(3))  
    binary = morphology.remove_small_holes(binary, area_threshold=100)  
  
    labels = measure.label(binary)  
    regions = measure.regionprops(labels, intensity_image=enhanced)  
  
    if not regions:  
        return binary, enhanced, threshold  
  
    # Prefer a compact bright component instead of the skull/background  
    scores = []  
    h, w = image.shape  
    for r in regions:
```

```

        cy, cx = r.centroid
        center_score = 1.0 - 0.7 * min(
            1.0,
            np.sqrt(((cy - h*0.50)/h)**2 + ((cx - w*0.50)/w)**2) * 2.5
        )
        score = r.area * max(r.mean_intensity, 1e-6) * max(center_score,
0.15)

        scores.append(score)

    best = regions[int(np.argmax(scores))]
    mask = labels == best.label

    mask = morphology.binary_closing(mask, disk(4))
    mask = morphology.remove_small_holes(mask, area_threshold=150)

    # Fallback for unusual intensity distributions
    if mask.sum() < image.size * 0.001:
        q = np.percentile(enhanced, 92)
        mask = enhanced > q
        mask = morphology.remove_small_objects(mask, min_size=40)
        mask = morphology.binary_closing(mask, disk(3))

    return mask, enhanced, threshold

tumor_mask, enhanced_img1, otsu_threshold = make_tumor_mask(img1)

# Appropriate structuring element
SE_RADIUS = 3
SE = disk(SE_RADIUS)

dilation = morphology.binary_dilation(tumor_mask, SE)
erosion = morphology.binary_erosion(tumor_mask, SE)

fig, ax = plt.subplots(1, 4, figsize=(18, 5))

ax[0].imshow(img1, cmap="gray")
ax[0].set_title("Input")
ax[0].axis("off")

ax[1].imshow(tumor_mask, cmap="gray")

```

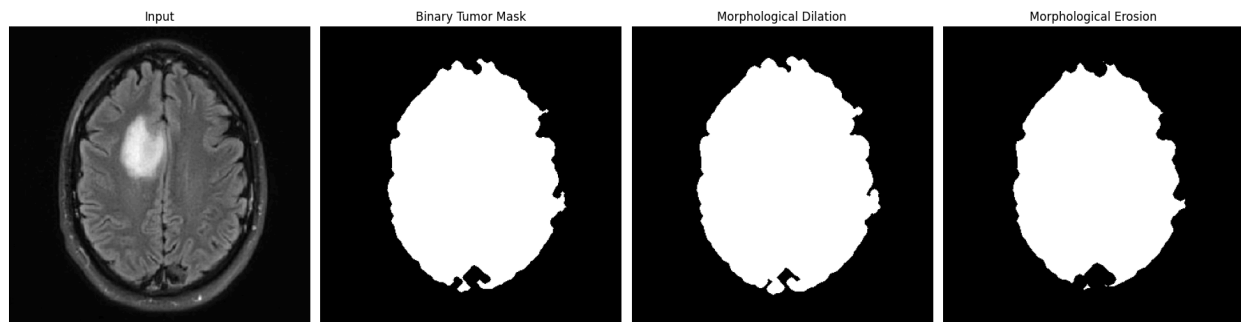
```
ax[1].set_title("Binary Tumor Mask")
ax[1].axis("off")

ax[2].imshow(dilation, cmap="gray")
ax[2].set_title("Morphological Dilation")
ax[2].axis("off")

ax[3].imshow(erosion, cmap="gray")
ax[3].set_title("Morphological Erosion")
ax[3].axis("off")

plt.tight_layout()
plt.show()

print("Otsu threshold:", round(float(otsu_threshold), 4))
print("Structuring element: disk(radius =", SE_RADIUS, ")")
```



2. Apply the Hough transform to Figure 2 and draw the detected lines on the original image.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage import util

# =====
# PROBLEM 2 - Canny + Probabilistic Hough Transform
# =====

xray_u8 = util.img_as_ubyte(img2)

smooth_xray = cv2.GaussianBlur(xray_u8, (5, 5), 0)

edges = cv2.Canny(smooth_xray, 50, 150)

min_len = max(30, int(min(img2.shape) * 0.08))

lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi / 180,
    threshold=50,
    minLineLength=min_len,
    maxLineGap=15
)
```

```

overlay = cv2.cvtColor(xray_u8, cv2.COLOR_GRAY2RGB)

chosen_lines = []

if lines is not None:

    line_info = []

    for line_segment in lines: # Fix: Iterate directly over lines (N, 4)
array
        x1, y1, x2, y2 = line_segment # Fix: Unpack coordinates directly
        length = float(np.hypot(x2 - x1, y2 - y1))
        angle = float(np.degrees(np.arctan2(y2 - y1, x2 - x1)))
        line_info.append((length, angle, (x1, y1, x2, y2)))

    line_info.sort(key=lambda z: z[0], reverse=True)

    chosen_lines = line_info[:50]

    for length, angle, (x1, y1, x2, y2) in chosen_lines:

        cv2.line(overlay, (x1, y1), (x2, y2), (255, 255, 255), 2)

fig, ax = plt.subplots(1, 3, figsize=(18, 6))

ax[0].imshow(img2, cmap="gray")

ax[0].set_title("Input X-Ray")

ax[0].axis("off")

ax[1].imshow(edges, cmap="gray")

ax[1].set_title("Canny Edge Map")

```

```
ax[1].axis("off")

ax[2].imshow(overlay)

ax[2].set_title("Hough Lines on Original")

ax[2].axis("off")

plt.tight_layout()

plt.show()

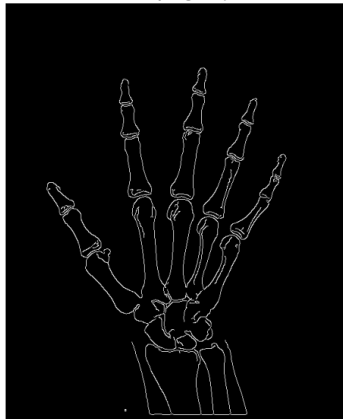
print("Raw Hough line segments:", 0 if lines is None else len(lines))

print("Displayed strongest segments:", len(chosen_lines))
```

Input X-Ray



Canny Edge Map



Hough Lines on Original



3. Segment the tumor from Figure 3 by using:

- i. Region growing approach**
- ii. Region Splitting and Merging approach**

```
# =====
# PROBLEM 3(i) - Region Growing
# =====

def automatic_seed(image):
    smoothed = filters.gaussian(image, sigma=2)
    h, w = image.shape

    work = smoothed.copy()
    my = max(4, int(0.08 * h))
    mx = max(4, int(0.08 * w))

    work[:my, :] = 0
    work[-my:, :] = 0
    work[:, :mx] = 0
    work[:, -mx:] = 0

    return np.unravel_index(np.argmax(work), work.shape)

def region_grow(image, seed, tolerance=0.07, max_fraction=0.12):
    image = image.astype(float)
    h, w = image.shape

    visited = np.zeros((h, w), dtype=bool)
    output = np.zeros((h, w), dtype=bool)

    neighbors = [
        (-1,-1), (-1,0), (-1,1),
        (0,-1),          (0,1),
        (1,-1),  (1,0),  (1,1)
    ]

    q = deque([seed])
    visited[seed] = True
    values = [image[seed]]
    max_pixels = max(100, int(h*w*max_fraction))
```

```

while q and output.sum() < max_pixels:
    y, x = q.popleft()
    mean_val = float(np.mean(values))

    if abs(image[y, x] - mean_val) <= tolerance:
        output[y, x] = True
        values.append(float(image[y, x]))

    for dy, dx in neighbors:
        ny, nx = y + dy, x + dx
        if 0 <= ny < h and 0 <= nx < w and not visited[ny, nx]:
            visited[ny, nx] = True
            q.append((ny, nx))

output = morphology.binary_closing(output, disk(2))
output = morphology.remove_small_holes(output, area_threshold=120)
output = morphology.remove_small_objects(output, min_size=50)

return output

seed = automatic_seed(img1)
region_growing_mask = region_grow(img1, seed, tolerance=0.075)

fig, ax = plt.subplots(1, 3, figsize=(16, 5))

ax[0].imshow(img1, cmap="gray")
ax[0].plot(seed[1], seed[0], marker="+", markersize=16, markeredgewidth=2)
ax[0].set_title("Input + Seed")
ax[0].axis("off")

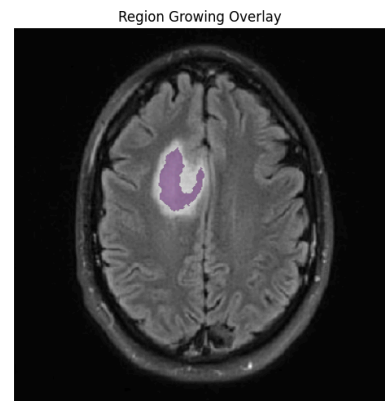
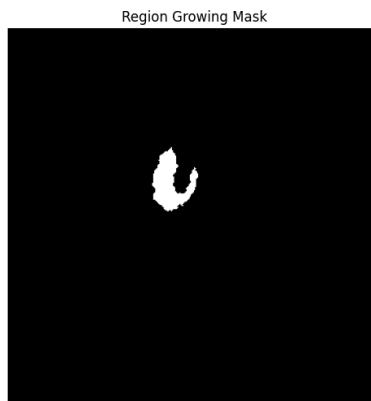
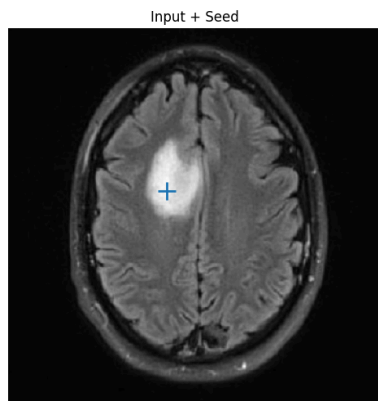
ax[1].imshow(region_growing_mask, cmap="gray")
ax[1].set_title("Region Growing Mask")
ax[1].axis("off")

ax[2].imshow(img1, cmap="gray")
ax[2].imshow(np.ma.masked_where(~region_growing_mask,
region_growing_mask), alpha=0.5)
ax[2].set_title("Region Growing Overlay")
ax[2].axis("off")

```

```
plt.tight_layout()
plt.show()

print("Seed (row, column):", seed)
```



```
# =====
# PROBLEM 3(ii) - Region Splitting and Merging
# =====

def split_quadtrees_regions(image, std_threshold=0.055, min_size=8,
max_depth=7):
    h, w = image.shape
    result = []

    def recurse(y0, y1, x0, x1, depth):
        block = image[y0:y1, x0:x1]
        if block.size == 0:
            return

        std = float(block.std())
        height = y1 - y0
        width = x1 - x0

        if (
            depth >= max_depth
            or min(height, width) <= min_size
            or std <= std_threshold
```

```

        ):
            result.append((y0, y1, x0, x1, float(block.mean()), std,
depth))

        return

    ym = (y0 + y1) // 2
    xm = (x0 + x1) // 2

    recurse(y0, ym, x0, xm, depth + 1)
    recurse(y0, ym, xm, x1, depth + 1)
    recurse(ym, y1, x0, xm, depth + 1)
    recurse(ym, y1, xm, x1, depth + 1)

recurse(0, h, 0, w, 0)
return result

split_regions = split_quadtree_regions(img1)

split_labels = np.zeros(img1.shape, dtype=np.int32)
for i, (y0, y1, x0, x1, mean_val, std_val, depth) in
enumerate(split_regions, 1):
    split_labels[y0:y1, x0:x1] = i

# Build adjacency from shared boundaries
def adjacency_pairs(labels):
    pairs = set()

    left = labels[:, :-1]
    right = labels[:, 1:]
    m = left != right
    for a, b in np.stack([left[m], right[m]], axis=1):
        if a > 0 and b > 0:
            pairs.add(tuple(sorted((int(a), int(b)))))

    top = labels[:-1, :]
    bottom = labels[1:, :]
    m = top != bottom
    for a, b in np.stack([top[m], bottom[m]], axis=1):
        if a > 0 and b > 0:
            pairs.add(tuple(sorted((int(a), int(b)))))

```

```

        return pairs

pairs = adjacency_pairs(split_labels)
means = np.array([r[4] for r in split_regions])

# Union-find merging
parent = np.arange(len(split_regions) + 1)

def root(a):
    while parent[a] != a:
        parent[a] = parent[parent[a]]
        a = parent[a]
    return a

def unite(a, b):
    ra, rb = root(a), root(b)
    if ra != rb:
        parent[rb] = ra

MERGE_THRESHOLD = 0.035

for a, b in pairs:
    if abs(means[a-1] - means[b-1]) <= MERGE_THRESHOLD:
        unite(a, b)

merged_labels = np.zeros_like(split_labels)
root_map = {}
new_id = 1

for old_id in range(1, len(split_regions) + 1):
    r = root(old_id)
    if r not in root_map:
        root_map[r] = new_id
        new_id += 1
    merged_labels[split_labels == old_id] = root_map[r]

tumor_merged_label = merged_labels[seed]
splitting_merging_mask = merged_labels == tumor_merged_label

```

```

splitting_merging_mask = morphology.binary_closing(
    splitting_merging_mask, disk(2)
)
splitting_merging_mask = morphology.remove_small_holes(
    splitting_merging_mask, area_threshold=100
)

fig, ax = plt.subplots(1, 4, figsize=(20, 5))

ax[0].imshow(img1, cmap="gray")
ax[0].set_title("Input")
ax[0].axis("off")

ax[1].imshow(split_labels, cmap="nipy_spectral")
ax[1].set_title(f"After Splitting ({len(split_regions)} regions)")
ax[1].axis("off")

ax[2].imshow(merged_labels, cmap="nipy_spectral")
ax[2].set_title(f"After Merging ({new_id-1} regions)")
ax[2].axis("off")

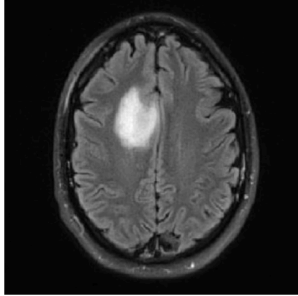
ax[3].imshow(img1, cmap="gray")
ax[3].imshow(
    np.ma.masked_where(
        ~splitting_merging_mask, splitting_merging_mask
    ),
    alpha=0.5
)
ax[3].set_title("Tumor Segmentation")
ax[3].axis("off")

plt.tight_layout()
plt.show()

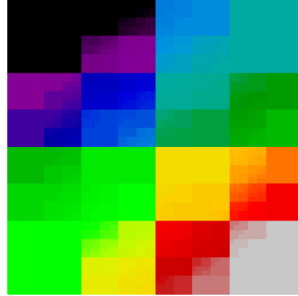
print("Split regions:", len(split_regions))
print("Merged regions:", new_id - 1)

```

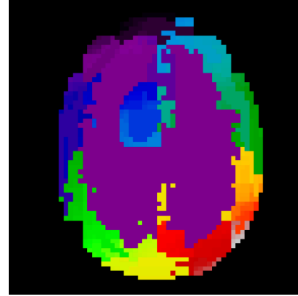
Input



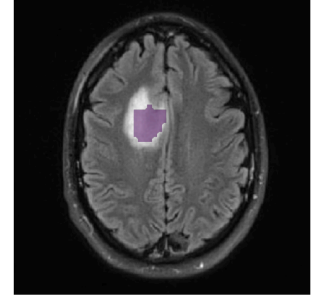
After Splitting (1240 regions)



After Merging (463 regions)



Tumor Segmentation



4. Segment the tumor from Figure 1 by using Marker Controlled Watershed segmentation.

```
# =====
# PROBLEM 4 - Marker-Controlled Watershed
# =====
smooth = filters.gaussian(img1, sigma=1.2)

# Bright candidate regions
bright = smooth > np.percentile(smooth, 90)
bright = morphology.binary_opening(bright, disk(2))
bright = morphology.binary_closing(bright, disk(3))
bright = morphology.remove_small_objects(
    bright, min_size=max(30, int(img1.size * 0.0002))
)

bright_labels = measure.label(bright)
seed_label = bright_labels[seed]

if seed_label != 0:
    foreground_marker = bright_labels == seed_label
else:
    foreground_marker = np.zeros_like(img1, dtype=bool)
    yy, xx = np.ogrid[:img1.shape[0], :img1.shape[1]]
    radius = 8
    foreground_marker = (
        (yy - seed[0])**2 + (xx - seed[1])**2 <= radius**2
    )

foreground_marker = morphology.binary_dilation(
    foreground_marker, disk(2)
)

background_marker = np.zeros_like(img1, dtype=bool)
background_marker[:5, :] = True
background_marker[-5:, :] = True
background_marker[:, :5] = True
background_marker[:, -5:] = True
background_marker |= smooth < np.percentile(smooth, 20)
```

```
background_marker &= ~foreground_marker

markers = np.zeros_like(img1, dtype=np.int32)
markers[background_marker] = 1
markers[foreground_marker] = 2

gradient = filters.sobel(smooth)
watershed_labels = segmentation.watershed(gradient, markers)

watershed_mask = watershed_labels == 2
watershed_mask = morphology.binary_closing(watershed_mask, disk(2))
watershed_mask = morphology.remove_small_holes(
    watershed_mask, area_threshold=100
)

fig, ax = plt.subplots(1, 5, figsize=(22, 5))

ax[0].imshow(img1, cmap="gray")
ax[0].set_title("Input")
ax[0].axis("off")

ax[1].imshow(foreground_marker, cmap="gray")
ax[1].set_title("Foreground Marker")
ax[1].axis("off")

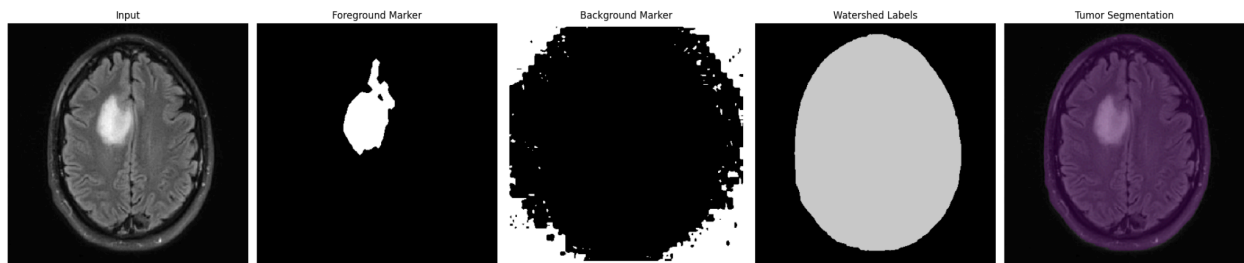
ax[2].imshow(background_marker, cmap="gray")
ax[2].set_title("Background Marker")
ax[2].axis("off")

ax[3].imshow(watershed_labels, cmap="nipy_spectral")
ax[3].set_title("Watershed Labels")
ax[3].axis("off")

ax[4].imshow(img1, cmap="gray")
ax[4].imshow(np.ma.masked_where(~watershed_mask, watershed_mask),
alpha=0.5)
ax[4].set_title("Tumor Segmentation")
ax[4].axis("off")

plt.tight_layout()
```

```
plt.show()
```



5. Segment the tumor from Figure 1 by using Quadtree Segmentation.

```
# =====
# PROBLEM 5 - Quadtree Segmentation
# =====

quadtree = split_quadtree_regions(
    img1,
    std_threshold=0.045,
    min_size=8,
    max_depth=8
)

# Boundary visualization
qt_boundary = np.dstack([img1, img1, img1])

for y0, y1, x0, x1, mean_val, std_val, depth in quadtree:
    cv2.rectangle(
        qt_boundary,
        (x0, y0),
        (max(x0, x1-1), max(y0, y1-1)),
        (1.0, 1.0, 1.0),
        1
    )

mean_threshold = np.percentile(
    [r[4] for r in quadtree],
    80
)

quadtree_mask = np.zeros_like(img1, dtype=bool)

for y0, y1, x0, x1, mean_val, std_val, depth in quadtree:
    cy = (y0 + y1) / 2
    cx = (x0 + x1) / 2

    normalized_distance = np.sqrt(
        ((cy - seed[0]) / img1.shape[0])**2 +
        ((cx - seed[1]) / img1.shape[1])**2
    )
```

```

        if mean_val >= mean_threshold and normalized_distance < 0.20:
            quadtree_mask[y0:y1, x0:x1] = True

quadtree_mask = morphology.binary_opening(quadtree_mask, disk(1))
quadtree_mask = morphology.binary_closing(quadtree_mask, disk(2))
quadtree_mask = morphology.remove_small_objects(
    quadtree_mask, min_size=40
)
quadtree_mask = morphology.remove_small_holes(
    quadtree_mask, area_threshold=100
)

fig, ax = plt.subplots(1, 3, figsize=(18, 6))

ax[0].imshow(img1, cmap="gray")
ax[0].set_title("Input")
ax[0].axis("off")

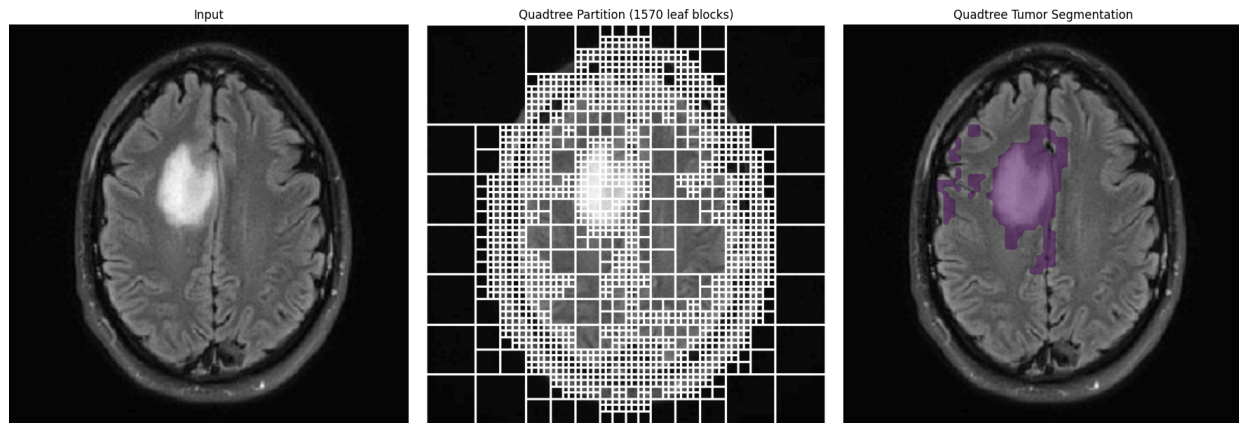
ax[1].imshow(quadtree_boundary)
ax[1].set_title(f"Quadtree Partition ({len(quadtree)} leaf blocks)")
ax[1].axis("off")

ax[2].imshow(img1, cmap="gray")
ax[2].imshow(np.ma.masked_where(~quadtree_mask, quadtree_mask), alpha=0.5)
ax[2].set_title("Quadtree Tumor Segmentation")
ax[2].axis("off")

plt.tight_layout()
plt.show()

print("Number of quadtree leaf blocks:", len(quadtree))

```



```
# =====
# FINAL - Side-by-side summary
# =====

fig, ax = plt.subplots(2, 5, figsize=(23, 10))

# Row 1: Problem 1 + Problem 2
ax[0,0].imshow(img1, cmap="gray")
ax[0,0].set_title("Input - Tumor")

ax[0,1].imshow(tumor_mask, cmap="gray")
ax[0,1].set_title("P1 - Binary Mask")

ax[0,2].imshow(dilation, cmap="gray")
ax[0,2].set_title("P1 - Dilation")

ax[0,3].imshow(erosion, cmap="gray")
ax[0,3].set_title("P1 - Erosion")

ax[0,4].imshow(overlay)
ax[0,4].set_title("P2 - Hough Transform")

# Row 2: Problems 3-5
ax[1,0].imshow(img1, cmap="gray")
ax[1,0].imshow(
    np.ma.masked_where(
        ~region_growing_mask, region_growing_mask
```

```

    ),
    alpha=0.5
)
ax[1,0].set_title("P3(i) - Region Growing")

ax[1,1].imshow(img1, cmap="gray")
ax[1,1].imshow(
    np.ma.masked_where(
        ~splitting_merging_mask, splitting_merging_mask
    ),
    alpha=0.5
)
ax[1,1].set_title("P3(ii) - Split + Merge")

ax[1,2].imshow(img1, cmap="gray")
ax[1,2].imshow(
    np.ma.masked_where(
        ~watershed_mask, watershed_mask
    ),
    alpha=0.5
)
ax[1,2].set_title("P4 - Marker Watershed")

ax[1,3].imshow(img1, cmap="gray")
ax[1,3].imshow(
    np.ma.masked_where(
        ~quadtree_mask, quadtree_mask
    ),
    alpha=0.5
)
ax[1,3].set_title("P5 - Quadtree")

ax[1,4].imshow(img2, cmap="gray")
ax[1,4].set_title("Input - X-Ray")

for a in ax.ravel():
    a.axis("off")

plt.tight_layout()
summary_path = OUT_DIR / "00_final_side_by_side.png"

```

```
plt.savefig(summary_path, dpi=200, bbox_inches="tight")
plt.show()

print("Final comparison saved to:", summary_path)
```

